Towards the Understanding and Evolution of Monolithic Applications as Microservices

Daniel Escobar, Diana Cárdenas, Rolando Amarillo, Eddie Castro,
Kelly Garcés, Carlos Parra, Rubby Casallas
Universidad de Los Andes
School of Engineering
Department of Systems and Computing Engineering
Bogotá, Colombia
email:{dan-esco, d.cardenas10, ra.amarillo71, ej.castro10,
kj.garces971, carlos.parra, rcasalla}
@uniandes.edu.co

***Abstract*—Nowadays, software development teams face the challenge of maintenance and evolution of large applications. This is known as *modernization* and comprises a process of *understanding* the existing application, and a process of *transforming* from the as-is state to the to-be state. In this article, we present an approach to modernize legacy applications into microservices. We propose a model-centered process to analyze and visualize the current structure and dependencies between the business layer and the data layer. Furthermore, we propose a modularization into clusters and microservices using four different diagrams. To validate our approach we have analyzed a JEE application with over 74566 LoC. The results show that the understanding is greatly improved through the diagrams proposed, and the modularization proposed constitutes the first step towards the automated modernization of the application.**

***Keywords*—Software Architecture, Software Reengineering, Modernization, Model-Driven Development, Microservices**

I. Introduction

Software is in constant change, not only to support the ever-changing requirements of clients but also to adapt to changes in technology. This type of adaptation is known as *Modernization*, which consists of two phases: 1) *Understanding* the existing system; 2) *Transforming* from the as-is state to the to-be state.

The way of understanding and transforming depends on the source and target technologies. In our case, the source is a given monolithic system, and the target is an application that offers the same business value that the monolithic one but using a microservice architecture.

A monolithic application does many things and has many responsibilities. Monolithic architectures are typically difficult to deploy, difficult to upgrade and maintain, and difficult to understand. Although not all monolithic systems have these problems, the reasons systems tend to get to this point are linked to fast changes in technology. These changes can make systems brittle, difficult and error prone, supporting many different technologies[1].

The microservice architectural style is fine-grained Software Oriented Architecture (SOA) due to the services that work together are *small and autonomous* [2]. The decomposition in small services makes the application less coupled by following the Single Responsibility Principle (SRP). This principle states that *each module, subsystem, class or method should not have more than one reason to change*[3]. This means that if we have different components each one must have an independent responsibility and need.

The modernization of monolithic systems into microservices is difficult because one has to decide how to distribute the legacy artifacts into microservices and establish the dependencies among them in order to preserve the original functionality. This distribution is not trivial since developers likely face legacy code with the following problems: 1) lack of design and documentation; 2) absent reasoning on the original design and architecture decisions; 3) undermining of the original design decisions as many additions and alterations have been made [4].

Understanding and transforming are needed for dealing with these problems. This work presents a proposal for the understanding stage, the transformation stage is out of scope. It is worth noting that the understanding stage is driven by keeping in mind the target, which is a microservice architecture. Therefore, we propose a process that takes a legacy application and produces a set of diagrams that help developers understand the legacy and suggests partitions of code in microservices. Developers can use these distributions as a starting point to make decisions about how to perform the actual transformation. We evaluate our approach through a case study where the legacy applications are made on top of the Java Enterprise Edition (JEE) platform.

At the core of our approach is the idea of separating and grouping Enterprise Java Beans (EJBs) according to the data they manipulate (e.g. separation by data [1]). We define an algorithm that evaluates the coupling of each EJB with each other within the application regarding the entities manipulated in the data tier. Using this analysis, we are able to suggest an initial proposal of `microservices` , where each one contains one or more EJBs. Model-driven techniques leverage the whole process and the construction of the diagrams.

The remainder of this paper is organized as follows. Section II presents the context, the running example, and the main research challenges. Section III presents our approach in

978-1-5090-1633-4/16/$31.00 2016 IEEE

detail. Section IV elaborates on the results of applying of our approach to a real JEE application. Section VI discusses the advantages and limitations of the proposal. Finally, in Section V we discuss the related work and Section VII presents the conclusions and points out new challenges for future work.

II. Problem Statement

Here we present the present a brief context on JEE applications, the main challenges faced in this research, and the running example used to illustrate the approach throughout the article.

A. JEE architecture

Java Enterprise Edition (JEE) is commonly used in enterprise applications. A JEE architecture has the following three tiers: presentation tier, business logic tier, and persistence layer.

- *The presentation tier* represents the front-end of the system. It transfers the user request to the Business logic tier for processing and retrieves the result from there and displays it on the screen. Usually this tier is developed using Java Server Pages (JSP), Java Server Faces (JSF) or plain servlets.
- *The business logic tier* houses all the major data processing functionalities of the system. It interconnects the presentation tier with the persistence tier. The Business logic tier fetches the results and sends them to the user. This tier contains the application logic encapsulated in different Enterprise Java Beans (EJB): session, message, managed, and entity beans. In this approach we will focus only in the use of session and entity beans. Session beans constitute the business layer and offer services while entity beans represent table in a relational database and are the bridge to the persistence tier.
- *The persistence tier* provides the abstraction for database access and thus separates it from other tiers. This task is usually done by a Java Persistence API (JPA) implementation (e.g. Eclipselink, Hibernate, etc).

B. Research Challenges

In order to modernize applications using the microservice architecture, it is necessary to understand the current architecture, and eventually perform a partition. In our case, we have chosen to provide a separation based on a *partition by data*[1] strategy. In the context of JEE, it means to consider the entity beans which represent the data and their relationships to the session beans in the business tier. This means that, for now, the presentation tier is out of the scope of our work.

To achieve this separation, we identify the following two challenges:

1) **Visualize highly/loosely coupled elements:** The approach has to provide an overview of the code elements contained in the business and persistence tiers (i.e., entity beans, session beans, and utility classes) and its dependencies. It should also identify and visualize highly and loosely coupled elements.

2) **Microservice distribution:** Using the information gathered in the previous item, it would be desirable to suggest a partition of beans into microservices. Highly coupled elements should be grouped together whereas loose coupled elements could be isolated and presented as independent microservices. It is also important to provide the list of dependencies (i.e., invocations) between the microservices. This would help to decide whether to partition or not a given set of code elements.

C. Running example: Marketplace

Our running example is a small marketplace application, which allows clients to purchase items from different products using a shopping cart. The products can be created, read, updated, and deleted. This application was generated using the *crudmaker* [5], that is a generator of JEE applications with CRUD functionalities. This application has 1560 LoC (lines of code) arranged in 35 folders, with a total of 77 classes. In addition, this project contains 156 methods. We use this running example to illustrate the different diagrams proposed.

III. Solution Overview

In order to face the challenges of visualization and microservice distribution presented in section II, we propose a *model-centered* approach based on three main phases as depicted in Figure 1: i) data injection, ii) querying, and iii) visualization. The approach is centered in views and aims at achieving a high-level understanding of the application. Two types of diagrams are proposed: (1) EJB-Clustering, and microservices. The former aims at presenting the application in terms of clusters, that are sets of types (e.g. session beans, data entities, and utility classes). The latter aims at proposing a groupment of clusters (i.e microservices) and the relationships between these microservices through invocations.

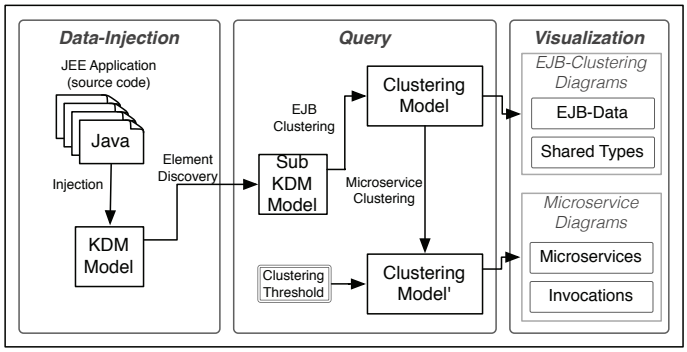

Fig. 1. Overall approach.

A. Data injection

The first step in the process consists of obtaining a model representation of the application. We use `MODISCO` [6] to process the source code of a JEE application. The result is a model with a complete Abstract Syntax Tree (AST) of the application that conforms to the KDM metamodel designed for

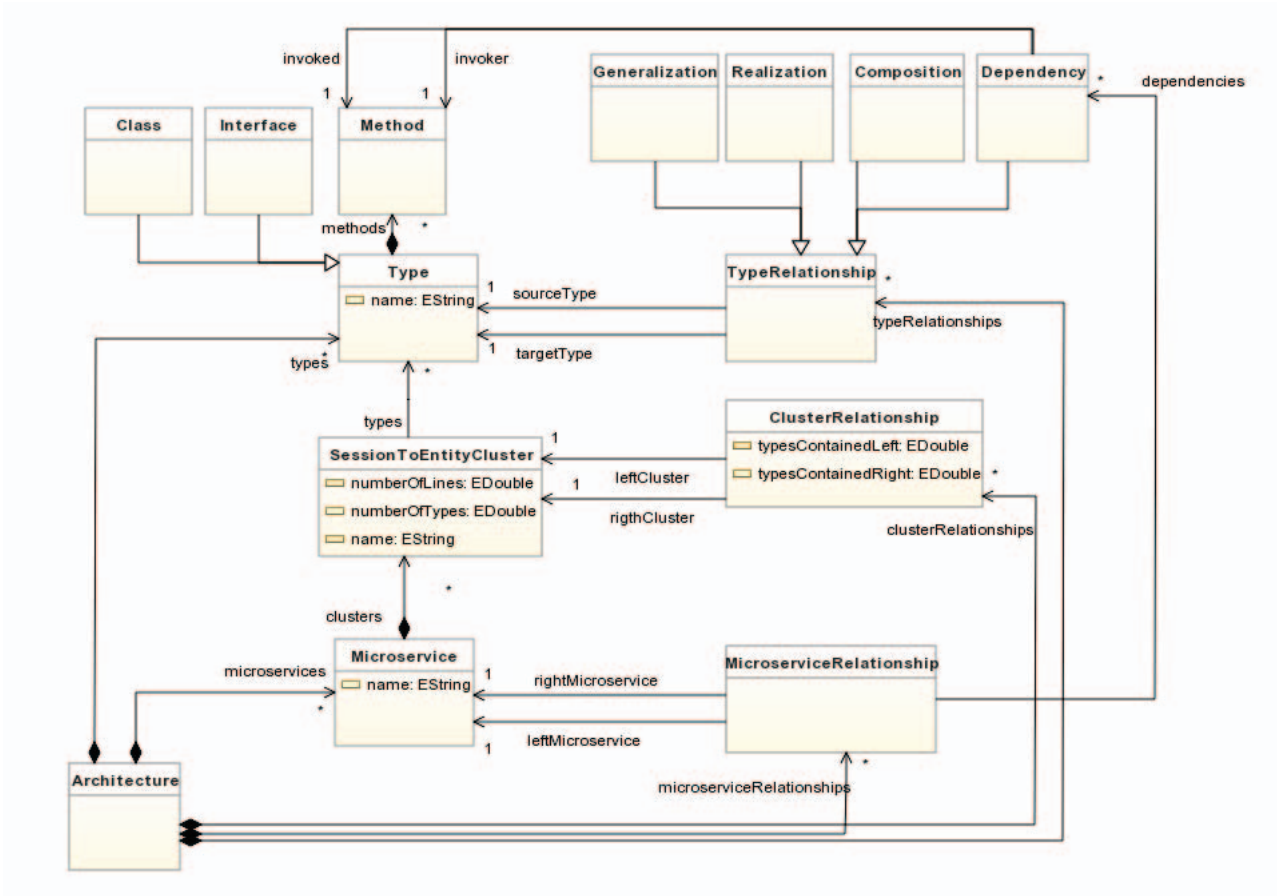

Fig. 2. Clustering architecture metamodel

the Eclipse JDT Java API. This model is used as input for the querying phase described below.

B. Querying

The second step in the approach is the querying, whose goal is to filter and transform the elements of the KDM model previously mentioned. The query starts with an *Element Discovery* phase that consists of finding the most relevant elements to the process such as types (e.g. classes, interfaces) with their attributes and method signatures, and the relationships between those types. The elements resulting from the query are represented in a simplified version of the KDM model, that we call Sub-KDM. After we obtain this model, we proceed with two different transformations towards the same clustering metamodel. Unlike the KDM and Sub-KDM metamodels that are verbose and describe the Java application in detail, the clustering metamodel contains only the elements that enable us to describe and propose a modularization of the application using the notions of cluster and microservice. Next, we describe the main concepts of the clustering metamodel (see Fig. 2).

- `Architecture` is the root element of the metamodel.
- `Type` describes Java code elements, i.e., `Classes` and `Interfaces`.
- `Class and Interface` are two types defined in the Java language.
- `Method` represents methods declared within a Type.
- `TypeRelationship` describes the types of relationships that can be established between types, that is, *generalization*, *realization*, *composition* and *dependency*. These relationships are obtained from a set of elements in the Java model elements as follows:
  1) A Generalization/Realization is established between A and B if B is a superclass/interface of A.
  2) A Containment is established between A and B if A has an attribute of type B.
  3) A Dependency is established between A and B in two cases: 1) if A calls a method of B or 2) A has a method that has an argument/return of type B.
- `SessionToEntityCluster` groups a set of Types which can be: 1) a session bean, 2) entities (data accessed by the logic in the session bean), 3) their superclasses/interfaces and 4) classes that connect the session bean and the entities in an indirect manner. This element has two attributes for the number of contained types and the total number of LoC.
- `ClusterRelationship` represents that two `SessionToEntityCluster` elements, referred to

as left and right, have one or more shared Types. The attribute `typesContainedLeft` is the percentage of types of a left-hand cluster that are contained in a right-hand cluster. Conversely, the attribute `typesContainedRight` is the percentage of types of a right-hand cluster that are contained in a left-hand cluster. In our approach, these percentages are useful to group `SessionToEntityClusters` into Microservices as explained in Section III-B1.

- `Microservice` is an element that groups `SessionToEntityCluster` elements implementing highly-cohesive services.
- `MicroserviceRelationship` represents the relationship between a pair of microservices. A relationship between microservices means that the left-hand side microservice contains at least one class whose implementation has a dependency (a method invocation) to a class inside the right-hand side microservice.

1) Clustering Transformations: Using the clustering metamodel as target, we define two transformations from the elements found in the Sub-KDM model: EJB Clustering, and Microservice Clustering.

a) EJB Clustering: The EJB clustering transformation is based on an algorithm that classifies the different types found in the Sub-KDM model inside clusters. All the types that process the same entities are grouped together. To achieve this, we have defined an algorithm that produces a graph. In the graph, the nodes represent types and the edges represent type relationships (e.g., generalization, realization, composition and dependency).

The algorithm selects pairs of nodes (A, B) where A is an EJB and B is an entity bean, and calculates the inheritance tree of A and B. After that, the algorithm finds all the invocations that start in A (or any type in the inheritance tree of A) and finish in B to identify the types in between, as shown in part *a* of Figure 3. For example, between the $EJB1$ and the $Entity1$, the types found are $2, 5, 7,$ and 8. Next, for each EJB, the algorithm creates a cluster and associates to it all the types found in the path (see part *b* of 3). It is important to notice that there may be types that belong to two or more clusters, for instance the types 3 and 9, as well as the $Entity2$ are found in the clusters for the $EJB1$ and $EJB2$. Besides, the algorithm also groups the types that are related to an EJB but that are not connected to an entity, in this case the type 1. This allows us to include utility classes in the cluster that are linked to an EJB but not to the data tier.

Once the algorithm creates the clusters, it establishes relationships between each pair of clusters (C_l, C_r) representing the percentage of types in common. A cluster relationship has the attributes `typesContainedLeft` and `typesContainedRight` whose values are calculated as follows:

$$typesContainedLeft = \frac{|C_l \cap C_r|}{|C_l|} * 100$$

$$typesContainedRight = \frac{|C_l \cap C_r|}{|C_r|} * 100$$

Where:
$|C_l \cap C_r|$ is the number of elements that results of the intersection between the set of types of C_l and the set of types of C_r.
$|C_l|$ is the total number of types of C_l.
$|C_r|$ is the total number of types of C_r.

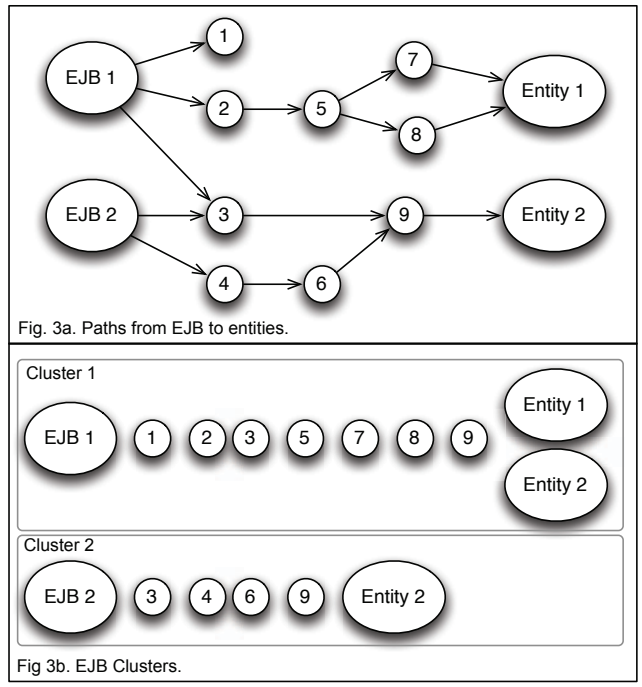

Fig. 3. EJB Clustering.

b) Microservice Clustering: The last step in the querying process groups different clusters according to the percentage of shared types. To achieve this, we define a *clustering threshold* that must be specified by the user. The algorithm navigates each C_l and validates if any of its cluster relationships has a percentage (e.g., values of typesContainedLeft or typesContainedRight attributes) higher than the *clustering threshold*. If so, the algorithm creates a Microservice M_i that groups the clusters C_l and C_r referenced by the property $rightCluster$ of the cluster relationship. The same validation is performed in C_r and its related clusters until all clusters are visited. At the end of the process, several microservice elements are created containing one or more clusters.

Once the clustering for EJBs and microservices is completed, the next phase of the process creates the views that show the information about clusters and microservices in a graphical way to help in the understanding and modularization of the JEE application.

C. Visualization

The last step in the process consists of several diagrams that illustrate the information obtained by the clustering process. Next, we describe the purpose and notation of each diagram.

1) EJB Clustering diagrams: These are two kinds of diagrams (i.e., EJB-Data and Shared Types) that target the first need for understanding stated out in Section 1, that is, coupling between elements.

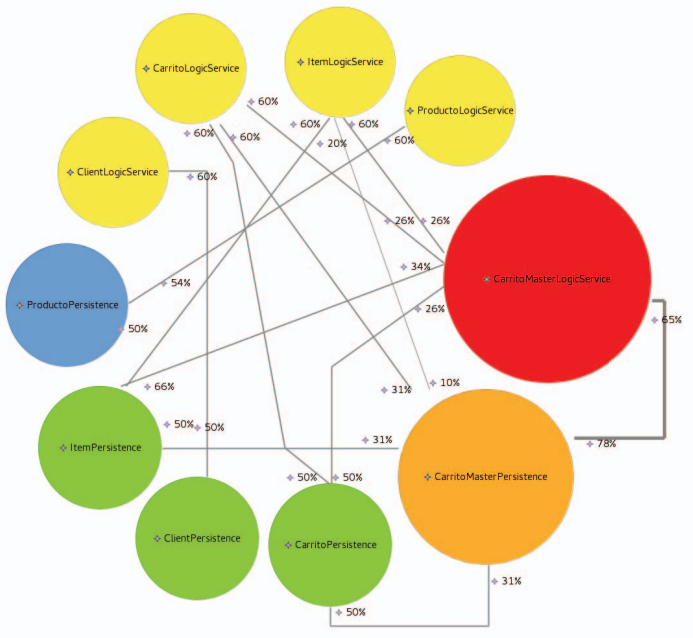

Fig. 4. Marketplace's EJB-Data diagram

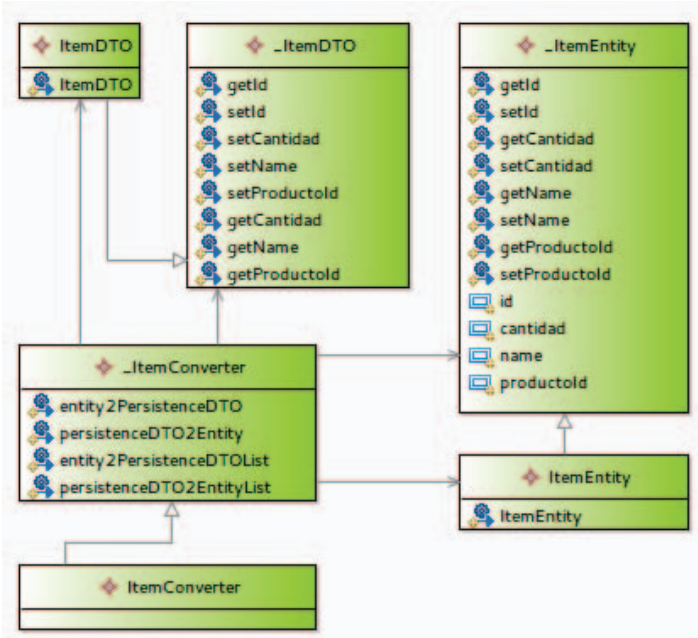

Fig. 5. Shared types between the ItemPersistence and CarritoMasterPersistence clusters

a) EJB-Data diagram: This diagram illustrates the paths and groups of types obtained by the EJB Clustering algorithm. The notation is explained below:

- A circle represents a `SessionToEntityCluster` and its label corresponds the name of the session bean contained in the cluster. The circle size is proportional to the number of types within the cluster. The color changes in a scale depending on the total number of LoC of every type. The possible colors are red, orange, gray, green, blue and yellow, where red is used for the cluster with the biggest number of LoC, and yellow is used for the cluster with the smallest number of LoC.
- A gray line represents a cluster relationship. The relationship ends have the percentages for the shared types between clusters. The line width is proportional to the number of relationships that exist between the types contained in the clusters A and B.
- The clusters are radially arranged in descending order by size. The cluster with the biggest size is positioned at three oclock and the remaining modules are organized proceeding clockwise.

Figure 4 shows the EJB-Data diagram for the running example. It contains a red circle that depicts a cluster for the `CartMasterLogicService` session bean, which is the largest regarding types contained and LoC. Also, the diagram represents that most of the cluster relationships originate in this cluster. From these two facts, we conclude that CartMasterLogicService is the session bean that consumes the most of the services provided by other session beans. In particular, the width and the percentages of the gray line going from CartMasterLogicService to CartMasterPersistence indicate a strong dependency between them. The reason for that is that most of CartMasterLogicService's functionalities access to the database via CartMasterPersistence.

b) Shared Types Diagram: The second diagram for the EJB clustering is a complement of the previous diagram and is generated by selecting a cluster relationship. It is similar to a UML class diagram and illustrates the shared Types between the clusters at each end of the relationship. It also illustrates the UML relationships between the types. The used notation is explained below:

- A rectangle represents a Type. The rectangle has two compartments for the type name, and the methods and attributes respectively.
- An arrow represents a type relationship. In particular, the arrow style describes the kind-of relationship: 1) a hollow triangle as an arrowhead specifies a Generalization; 2) an open arrowhead indicates a Dependency; 3) an arrowhead with a filled black diamond means a Composition.
- The diagram layout locates the types in a way that relationships intercept as little as possible.

Figure 5 shows the diagram obtained from the relationship between the clusters `ItemPersistence` and `CarritoMasterPersistence`. The diagram depicts all the shared types between the clusters (i.e., ItemEntity, ItemDTO, and ItemConverter), as well as the UML relationships between these types. These types represent the Item records stored in the database. ItemPersistence needs to know them because it has the logic to read/write/update/delete records of the Item database table. In turn, CarritoMasterPersistence requires to know them to associate items to a given cart.

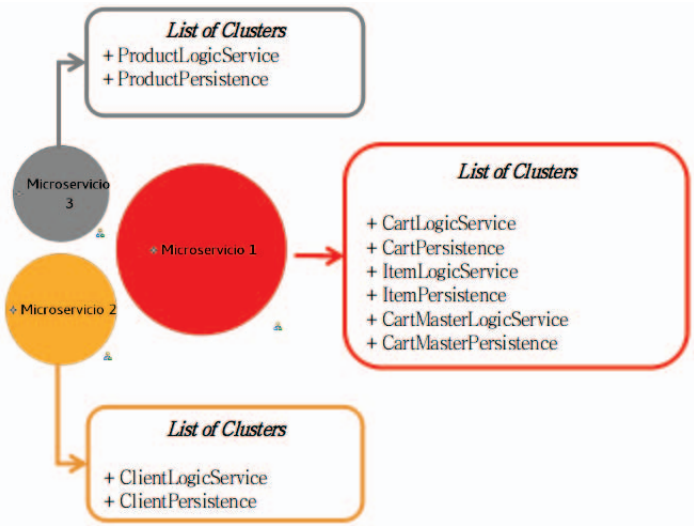

Fig. 6. Marketplace's microservices when using a clustering factor of 50%

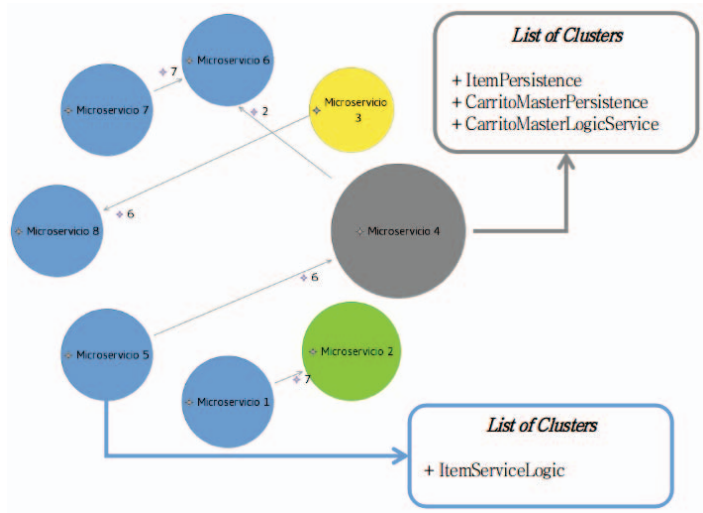

Fig. 7. Marketplace's microservices when using a clustering factor of 65%

2) Microservices diagrams: We have defined two diagrams to represent microservices and their invocations. The purpose of these diagrams is to achieve modularization, as stated in the second research challenge of section II-B.

a) Microservices diagram: This diagram proposes a separation in microservices that group clusters. The used notation is explained below:

- A circle represents a MicroService. The circle label is the microservice name, which is by default generated as an enumeration but that should be changed by the user into a more meaningful name. The size of the circle is proportional to the number of clusters contained. Similar to the cluster diagram, colors are used to represent the size of the microservice in terms of the number of types contained and the LoC of each type.
- A gray line describes a relationship between microservices and has a sense and a number. For example, a relationship that goes from a microservice A to B, represents method invocations originating from a class inside A and ending in a class inside B. The number indicates the amount of invocations found.
- Like in III-C1, the diagram layout is radial.

Figure 6 shows the Marketplace's microservices diagram, when using a clustering threshold of 50%. The Figure shows three microservices, and no relationships among the microservices which makes this distribution ideal with respect to quality attributes: 1) Requests can be built to access a particular microservices —deployed in a dedicated server— depending on the data processed (i.e., either Product, Client, and, Cart and Item), instead of sending all requests to a centralized server. This favors performance, scalability and availability; 2) In this case, each microservice is able to resolve each request by itself, that is, there is no need for calling other microservices. The latter also favors performance because the application saves the cost of forwarding the request among microservices; 3) The separation of functionalities benefits maintainability, every microservice can evolve independently of the rest of the application. It is worth noting that this distribution is obtained because the Marketplace application has been implemented by following design principles of low coupling and high cohesion.

Figure 7 shows the Marketplace's microservices when using a clustering threshold of 65%. On the one hand, This suggestion of distribution of Types is very similar to that of Figure 4, that is, most of microservices correspond to a cluster, except in the case of the Microservice 4 that groups three clusters. On the other hand, this suggestion has relationships and more microservices than those of Figure 6. Given that the microservices are more granular, deployment (and then maintenance) becomes more complex. In addition, more relationships result in more calls between microservices, which impacts performance. This distribution can be more complex to implement than that of Figure 6, however it achieves more granularity. It also illustrates how the relationships among microservices look like, and how the clustering threshold affects the resulting diagram.

b) Microservices invocation diagram: This is the final diagram and is available for each microservice relationship. The purpose for this diagram is to visualize the invocations between the types contained in each microservices. The used notation is explained below:

- A gray arrow represents method invocations. Each arrow has a direction which is from the `simple end` to the `end with an open arrowhead`. The simple end represents the session bean that consumes the methods. In turn, the end with an open arrowhead indicates the session bean that provides the called methods.
- A gray rectangle represents session beans, either provider or consumer. The first compartment of the rectangle shows: 1) the name of the microservice that contains the session bean; and 2) the name of the session bean. If the second compartment is empty, then the rectangle represents a provider. In contrast, if the second compartment spells out method names,

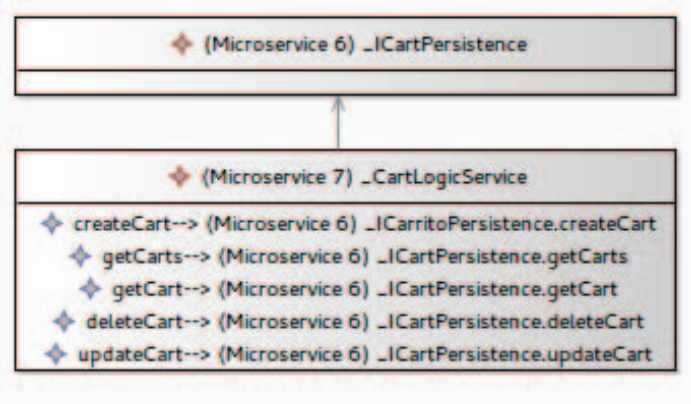

Fig. 8. Invocations between microservices 6 and 7

then, the rectangle represents a consumer. The method representation has the following convention: the name of the method from which the consumer performs the call, the name of the provider microservice, the name of the provider session bean, and the name of the called method.

- This diagram has the same layout than that of the shared types diagram.

Figure 8 shows the calls between the microservices 6 and 7 shown in Figure 7. From bottom to top, one encounters the calls from _CarritoLogicService to _ICarritoPersistence. As can be seen in the diagram, the type `ICartPersistence` from the $microservice6$ is a provider whose consumer is the type `CartLogicService` from the $microservice7$. The methods invoked are typical CRUD operations on the cart of the marketplace.

IV. Evaluation

The evaluation of our process involved three aspects: 1) know whether the process is useful for large applications; 2) outline some ideas about the understanding that can be achieved within the approach; and 3) calculate the performance of the tool. In this section, we characterize the dataset used in the evaluation, present the results and an analysis, and make the performance explicit.

A. Dataset

To validate our approach, we have analyzed a large JEE application so-called *SISINFO*. It manages most of the academic processes and information of the Department of Systems and Computing Engineering in *Universidad de los Andes*. This application is built on top of JEE version 6, and has 74566 LoC, 624 classes, and 35993 methods.

B. Results and Analysis

1) EJB-Data Diagrams: Figure 9 shows the SISINFO's EJB-Data Diagram. We present some metrics of this diagram in the first row of Table I. As can be seen in the table, the diagram has 172 clusters, and the number of types of each cluster varies from 3 to 78. The metrics of the table confirm the first impression when looking at the Figure, which is that most of the clusters (113) are coupled with each other. Only 59 clusters are loosely coupled (few or none relationships with each other), most of them will result in an isolated microservice. Finally, the cluster *WordLoadAndCommitmentsBean* given its size both in LoC and number of types contained, is the cluster with the lowest cohesion and then the most difficult to maintain.

TABLE I. Metrics obtained from the SISINFO's diagrams

Diagram	Metric	Result
EJB-Data	# of EJB Clusters	172
	# of EJB Clusters loosely coupled	59
	Minimum of Types per microservice	3
	Maximum of Types per microservice	78
	Largest EJB Cluster relation by LoC and Types	From *Tesis1Bean* to *TesisBean* (LoC: 1389 and Types: 70)
	Largest EJB Cluster by LoC and Types	*WordLoadAndCommitmentsBean* (LoC: 1691 and Types: 78 - Relations with other Clusters: 3)
Microservices	# of Microservices created	67
	# of Isolated Microservices	24
	Lowest # of types per microservice	3
	Highest # of types per microservice	405
	Largest consumer of microservices	*Microservice 1* (Dependencies: 45; LoC: 7105; Types: 405)

2) Microservices diagrams: We obtained different microservices diagrams by using 3 clustering thresholds, i.e., 5%, 50%, and 80%. In the microservice diagrams of 5% and 50%, we observed that the number of types contained in the microservices were very unequal, for example, in the diagram of 5%, 22 microservices had 1 type and 1 microservice had 152 types. The clustering threshold that gave the best proportion of number of types was 80% (see Figure 10). We summarized some metrics of the microservice diagram of 80% in the second row of Table I. As can be seen in the table, the diagram has 67 microservices, 24 of which are isolated. The largest consumer is *Microservice 1* (red circle) with a number of outgoing dependencies of 45. Also, the number of types per microservice ranges from 3 to 405.

From these metrics, we can conclude the following:

1) The number of types contained in the microservices for the 80% threshold is homogeneous, 62 microservices (yellow circles) had 3 types and 41 LoC in average. However, 5 microservices have 88 types and 1518 LoC in average. It would be easier to maintain the former group of microservices —which is the largest group— than the latter.
2) The isolated circles represent good candidates of microservices that can be deployed without the need to re-build and re-deploy the whole application.
3) Regarding the Microservice 1, which has the most of outgoing dependencies, it acts like a broker that

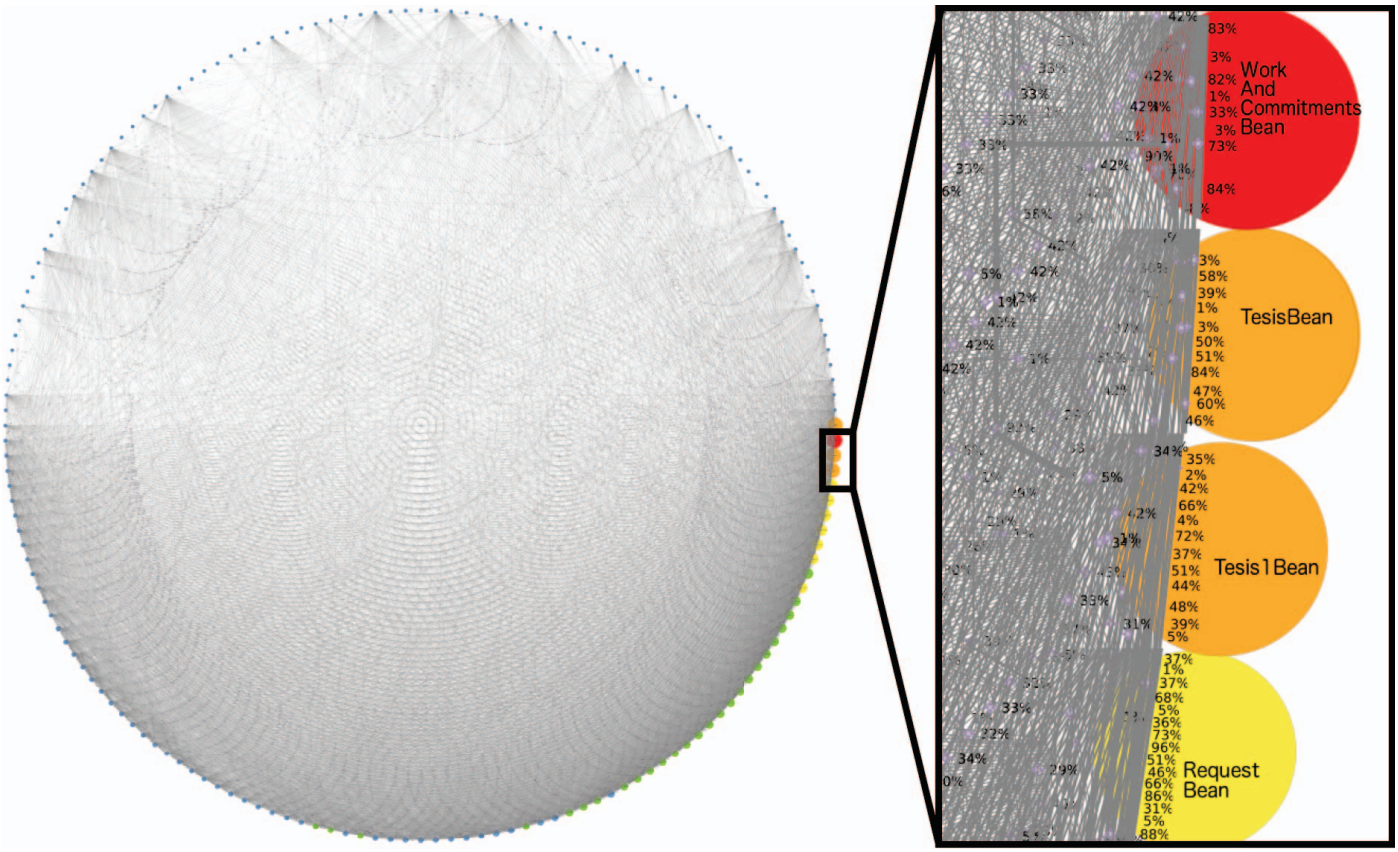

Fig. 9. SISINFO's EJB-Data Diagram

controls the access to most of the functionalities in the application. This indicates that the types inside this microservice are highly coupled, and might become a bottleneck or even a target for attacks on the application. In a nutshell, the SISINFO's microservices diagram may help analyze the (dis)advantages of the suggested distribution in terms of quality attributes such as maintainability, performance, and security.

Finally, Figure 11 depicts the relationship between *Microservice 1 and 60*, which is the link with the highest number of method invocations (i.e., 245). The figure shows a subset of the invocations and types involved, its purpose is to draw the reader's attention to the high number of invocations starting from the types inside the *Microservice 1* (e.g., *TesisBean, ProyectoDeGradoBean*) and ending at the *CorreoLocal* session bean of *Microservice 60*. Thus, the diagram may point to the inclusion of this bean into the *Microservice 1* if possible, to avoid forwarding numerous requests between the types.

C. Performance

The machine used for our evaluation had the following features: Windows 10 operative system, CPU of 8 cores and 8 GB of RAM. The tool produced the SISINFO's views in 11 hours, distributed as follows: 4 hours for data injection, 3 hours for query, and 4 hours for views generation. The reasons for these execution times are the verbosity of the KDM model provided by `MODISCO` and the performance offered by the clustering algorithms and Sirius. This could be improved by implementing a customized Java parser that focuses on the elements that matter, profiling the algorithms to aid their optimization, and using tools alternative to Sirius. Therefore, these improvements were left for further work.

V. Related work

In modernization, the understanding phase produces different artifacts presented in the form of graphics or text. Below, we distinguish approaches considering the type of output. Although there are many works that perform understanding in other contexts (e.g., software maintenance, reverse engineering, knowledge engineering), we restrict our literature review to approaches whose gist is software modernization:

A. Graphical output

The purpose of each graphic is to represent a concern of the original system in an intuitive way. Implicitly this means to reduce the amount of information in order to achieve comprehension. We found that the output notation used in related works can be classified in: i) standard or ii) domain specific. The second option is suggested over the first one in cases where the modernization includes experts/users to which a customized graphical notation results on straightforward comprehension and communication. However, the second option implies a higher development effort when compared with the first option: whereas the first option can reuse existing

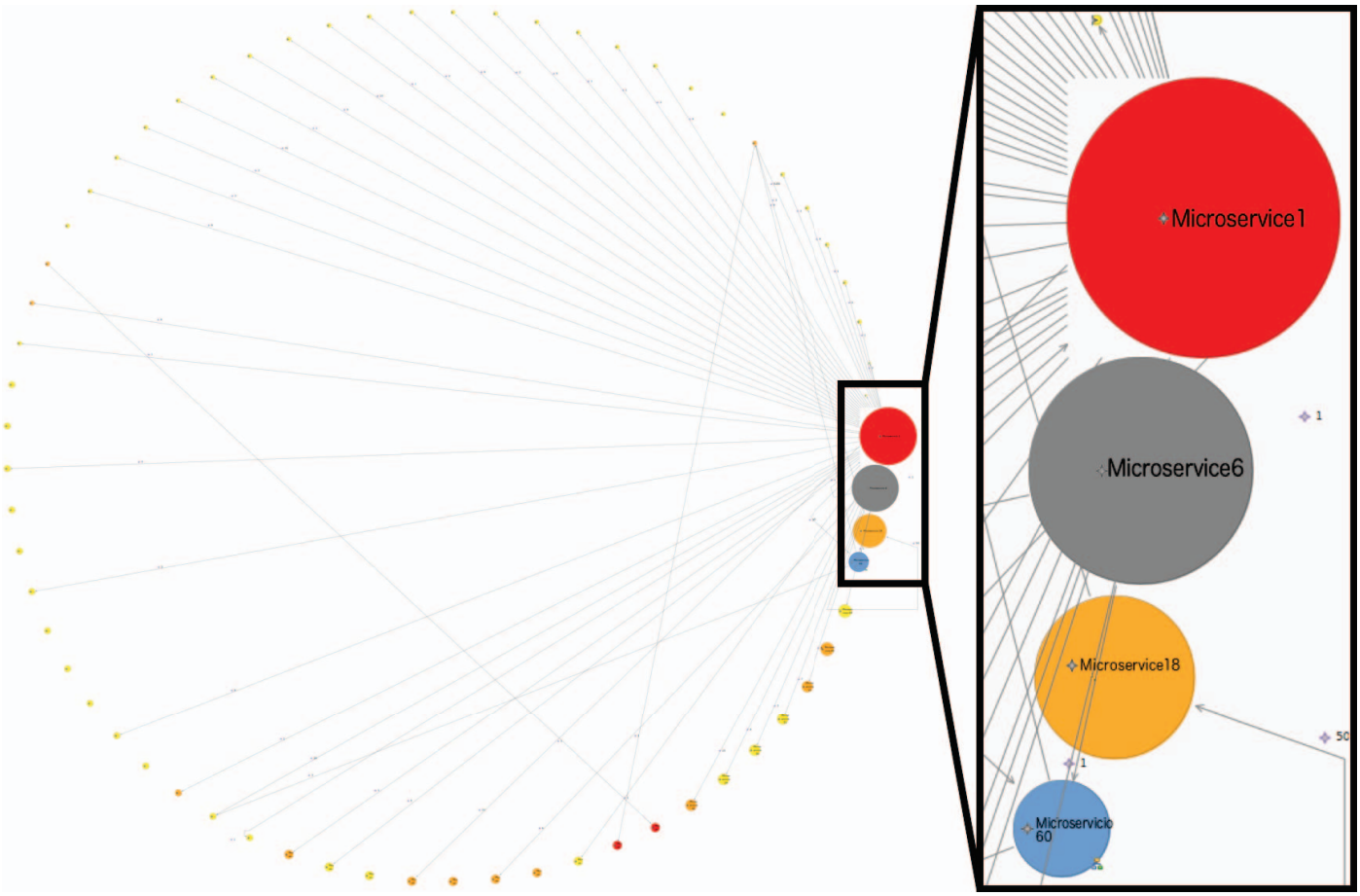

Fig. 10. SISINFO's Microservices Diagram

viewers, the second option often requires the construction of viewers from scratch. The standard notation in the articles [7], [8], [9] and the case studies presented in Chapters 5, 6, 8, 10-12, 14 of [10] is the Unified Modeling Language (UML) one. Other standard notation is graph theory where nodes and edges are generic enough to represent any kind of software element (e.g., classes, functions) and relationships between elements (e.g., inheritance, function call). An article that use graph notation is [11]. An additional standard notation encountered in related work is chart that displays quantitative information. For example, [12] uses bar charts to indicate the proportion of Oracle Forms triggers that contain a particular kind of coupling. In turn, Nagy et al. [13] propose domain specific notations.

B. Textual output

The purpose of textual output is to provide with detailed information about an aspect of the system. Text is harder to utilize than graphical outputs because software engineers simply drown in the flood of information. That is why, in many cases, engineers look at the graphics first, and then they complement their knowledge with the text. We found that two related works extract business rules from legacy systems and represent them in a textual manner. [14] use the rules at different stages of modernization, for example, when locating the code affected by a change in a rule. The rules looks like logical statements. In turn, the case study in Chapter 14 of [10] performs the extraction of rules to validate if they are still aligned with the organization policies. The rules are similar to pseudo-code.

C. Comparison

We compared our approach to related work and we obtained the following conclusions: the output of the understanding phase in our approach is graphical. Similarly to [13] and the case study in Chapter 9 of [10], we propose notations for the particular domain of JEE applications and microservices. There is a noticeable difference between our approach and related work: whereas the latter produces either visualizations or metrics, the former generates *metric-centered visualizations*, that is, numerical data transformed into visual representation. The purpose is to make easier understanding. A quite good number of documents on software visualization [15], [16], [17], [18], [19], [20] demonstrates that this concept is not new in maintenance or software mining context, but it is new in software modernization.

VI. Discussion

The main benefit of our approach is to suggest different distributions of a given code into microservices, regardless of the application size. Whereas classical code inspection

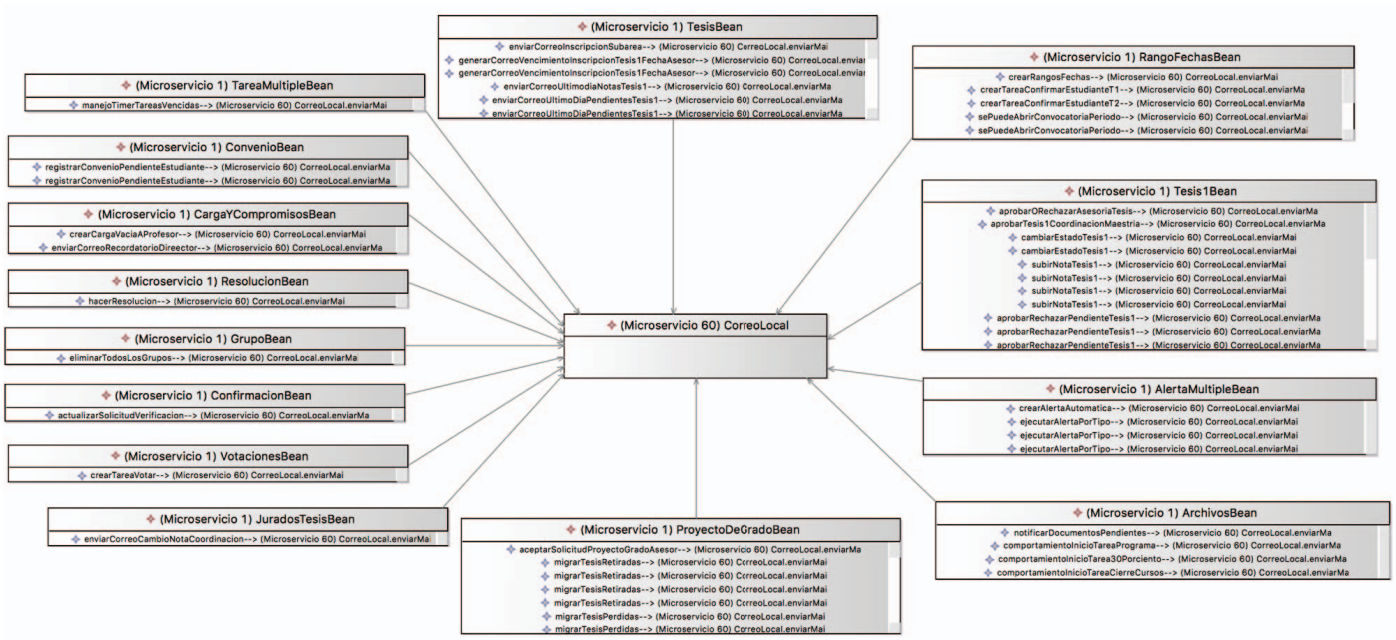

Fig. 11. Invocations between microservices 1 and 60 of SISINFO

gives a low-abstraction level information of the code and is time-consuming, our proposal produces high-abstraction level diagrams that allow the developers to figure out easily what parts of the code can be partitioned into microservices and what parts of the code have to be kept in coarse-grained services. From this, the developer can evaluate the distribution disadvantages and propose strategies to overcome them, for example, given the SISINFO's Microservice 1, he can decide between: 1) keeping the Microservice 1 code as is and maintain multiple replicas of it to avoid bottlenecks, or 2) refactoring the code to make it more modular. Although the final decision depends on additional restrictions such as the budget available to pay for redundant infrastructure or new developers that refactor the code, our proposal speeds the first step of modernization, which is understanding. However, it is important to notice that our approach also suffers from some limitations as briefly discussed below.

A. Static analysis restrictions

Currently, the understanding process uses the tools provided by `MODISCO`. Our approach only works for Java applications, and more specifically, multi-tier applications developed under the JEE specification, where business and data types are properly identified through Java annotations.

Furthermore, the approach uses a static analysis. As a consequence we are not able to identify dependencies between the business and data layers that are not specified through Java-annotations. A runtime analysis would overcome this limitation, by analysing every requests on the data base. However, this type of analysis is out of the scope of this paper.

B. Automated transformation

To fully modernize an application, it is important not only to understand the current state and propose a set of microservices, but also to actually transform the application by organizing the code according to the proposed distribution. More importantly, it is necessary to refactor the application, in order to create the services that have to be provided and required, as depicted in the microservice and invocation diagrams.

We envision an automated process similar to the one presented in [21] that generates the glue code to wrap classes and replaces direct EJB dependencies by exposing and consuming REST services. However, for the time being, this transformation process and its implementations is left as future work.

VII. Conclusions and Future Work

In this article we have presented an approach to tackle the challenges of modernization of legacy JEE applications using a static code analysis. Such challenges refer to the visualization of the current legacy JEE application, and the partition of the application into microservices. For the former one, we have defined and implemented two different diagrams from the source code, which can be used by architects and developers to apprehend the application architecture. For the latter one, we have defined a microservice diagram that provides some suggested options about how to split the legacy application in small pieces of code, called microservices. These diagrams can be a good starting point to take decisions about the evolution of large applications.

Our model-centered approach was applied to a *marketplace* application and *SISINFO*. The results of Section IV indicate that the EJB-data diagram improves the understanding of the application and help architects to analyze the relationship between the EJBs. Using this information, and allowing the user to provide a clustering threshold, the microservices diagram provides an initial separation by data, with a preferred

proportion regarding the number of microservices obtained and the size of each one.

For future work, we aim to improve the performance of our visual editor, in this moment it takes too much time to process, query, and generate the different views of a JEE application. Moreover, we want to create more diagrams that enable architects to have a better legacy application understanding. Furthermore, we want to build a fully automated modernization process, by implementing the transformation that reflects the distribution proposed by the microservice diagram. To achieve this, we plan to use the information of the microservice invocation diagram to generate REST wrappers that separate the source code and handle these invocations remotely.

VIII. Acknowledgements

We would like to thank César Espinosa and Diego Hoyos who improved the algorithms, implementation and diagrams presented in this article. We would also like to thank Luis Felipe Mendivelso and Mauricio Verano who have contributed in the preparation of this manuscript.

References

[1] L. Krause, Microservices: Patterns and Applications: Designing fine-grained services by applying patterns., 1st ed., June 2015.

[2] S. Newman, Building Microservices, 1st ed., M. Loukides and B. MacDonald, Eds. O'Reilly Media, Inc., February 2015.

[3] R. C. Martin. (2009, November) The single responsibility principle. [Online]. Available: http://programmer.97things.oreilly.com/wiki/index.php/The_Single_Responsibility_Principle

[4] T. C. Lethbridge and N. Anquetil, "Advances in software engineering," H. Erdogmus and O. Tanir, Eds. New York, NY, USA: Springer-Verlag New York, Inc., 2002, ch. Approaches to Clustering for Program Comprehension and Remodularization, pp. 137–157. [Online]. Available: http://dl.acm.org/citation.cfm?id=505630.505637

[5] TICSw-Uniandes. (2015) Crudmaker. [Online]. Available: https://sistemasacademico.uniandes.edu.co/~recursos-csw/dokuwiki/doku.php?id=generador:instalacion

[6] H. Bruneliere, J. Cabot, F. Jouault, and F. Madiot, "Modisco: A generic and extensible framework for model driven reverse engineering," in Proceedings of the IEEE/ACM International Conference on Automated Software Engineering, ser. ASE '10. New York, NY, USA: ACM, 2010, pp. 173–174. [Online]. Available: http://doi.acm.org/10.1145/1858996.1859032

[7] Y. Dajsuren, M. Goldstein, and D. Moshkovich, "Modernizing legacy software using a system grokking technology," in Software Maintenance (ICSM), 2010 IEEE International Conference on, Sept 2010, pp. 1–7.

[8] R. Pérez-Castillo, I. García-Rodríguez de Guzmán, I. Caballero, and M. Piattini, "Software modernization by recovering web services from legacy databases," Journal of Software: Evolution and Process, vol. 25, no. 5, pp. 507–533, 2013.

[9] A. Bergmayr, H. Bruneliere, J. Canovas Izquierdo, J. Gorronogoitia, G. Kousiouris, D. Kyriazis, P. Langer, A. Menychtas, L. Orue-Echevarria, C. Pezuela, and M. Wimmer, "Migrating legacy software to the cloud with artist," in Software Maintenance and Reengineering (CSMR), 2013 17th European Conference on, 2013, pp. 465–468.

[10] W. M. Ulrich and P. Newcomb, Information Systems Transformation: Architecture-Driven Modernization Case Studies. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2010.

[11] G. Boussaidi, A. Belle, S. Vaucher, and H. Mili, "Reconstructing architectural views from legacy systems," in Reverse Engineering (WCRE), 2012 19th Working Conference on, Oct 2012, pp. 345–354.

[12] J. Izquierdo and J. Molina, "An architecture-driven modernization tool for calculating metrics," IEEE Software, vol. 27, no. 4, pp. 37–43, 2010.

[13] C. Nagy, L. Vidács, R. Ferenc, T. Gyimóthy, F. Kocsis, and I. Kovács, "Solutions for reverse engineering 4gl applications, recovering the design of a logistical wholesale system," 2011, pp. 343–346.

[14] E. Putrycz and A. Kark, "Recovering business rules from legacy source code for system modernization," in Advances in Rule Interchange and Applications, ser. Lecture Notes in Computer Science, A. Paschke and Y. Biletskiy, Eds. Springer Berlin Heidelberg, 2007, vol. 4824, pp. 107–118.

[15] M. Lanza and S. Ducasse, "Understanding software evolution using a combination of software visualization and software metrics," in In Proceedings of LMO 2002 (Langages et Modeles a Objets. Lavoisier, 2002, pp. 135–149.

[16] L. Voinea and A. Telea, "Mining software repositories with cvsgrab," in Proceedings of the 2006 International Workshop on Mining Software Repositories, ser. MSR '06. New York, NY, USA: ACM, 2006, pp. 167–168. [Online]. Available: http://doi.acm.org/10.1145/1137983.1138024

[17] A. McNair, D. German, and J. Weber-Jahnke, "Visualizing software architecture evolution using change-sets," in Reverse Engineering, 2007. WCRE 2007. 14th Working Conference on, Oct 2007, pp. 130–139.

[18] M. DAmbros, H. Gall, M. Lanza, and M. Pinzger, "Analysing software repositories to understand software evolution," in Software Evolution. Springer Berlin Heidelberg, 2008, pp. 37–67.

[19] A. Gonzalez, R. Theron, A. Telea, and F. J. Garcia, "Combined visualization of structural and metric information for software evolution analysis," in Proceedings of the Joint International and Annual ERCIM Workshops on Principles of Software Evolution (IWPSE) and Software Evolution (Evol) Workshops, ser. IWPSE-Evol '09. New York, NY, USA: ACM, 2009, pp. 25–30.

[20] T. Khan, H. Barthel, A. Ebert, and P. Liggesmeyer, "Visualization and Evolution of Software Architectures," in Visualization of Large and Unstructured Data Sets: Applications in Geospatial Planning, Modeling and Engineering - Proceedings of IRTG 1131 Workshop 2011, ser. OpenAccess Series in Informatics (OASIcs), C. Garth, A. Middel, and H. Hagen, Eds., vol. 27. Dagstuhl, Germany: Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2012, pp. 25–42. [Online]. Available: http://drops.dagstuhl.de/opus/volltexte/2012/3739

[21] C. Parra and D. Joya, "SPLIT: An Automated Approach for Enterprise Product Line Adoption Through SOA," Journal of Internet Services and Information Security (JISIS), vol. 5, no. 1, pp. 29–52, February 2015.